OS-Integrated Buffer Management for High-Throughput Analytical Workloads

Guided Research Project

Maurice Scholtes
Advisor: Ilya Meignan--Masson
Systems Research Group
https://dse.in.tum.de/



20.04.2026 – 20.10.2026

Cloud database systems

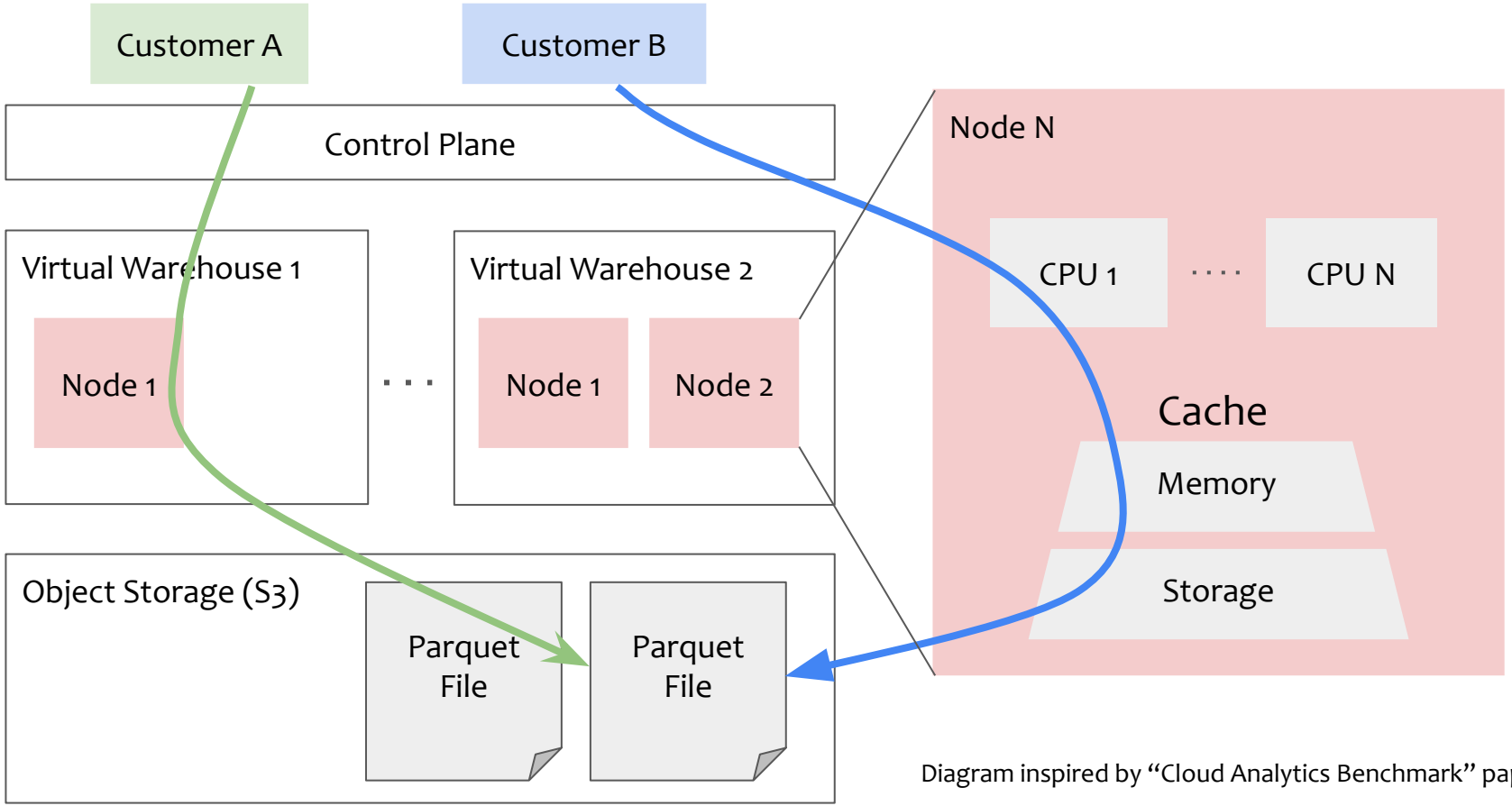
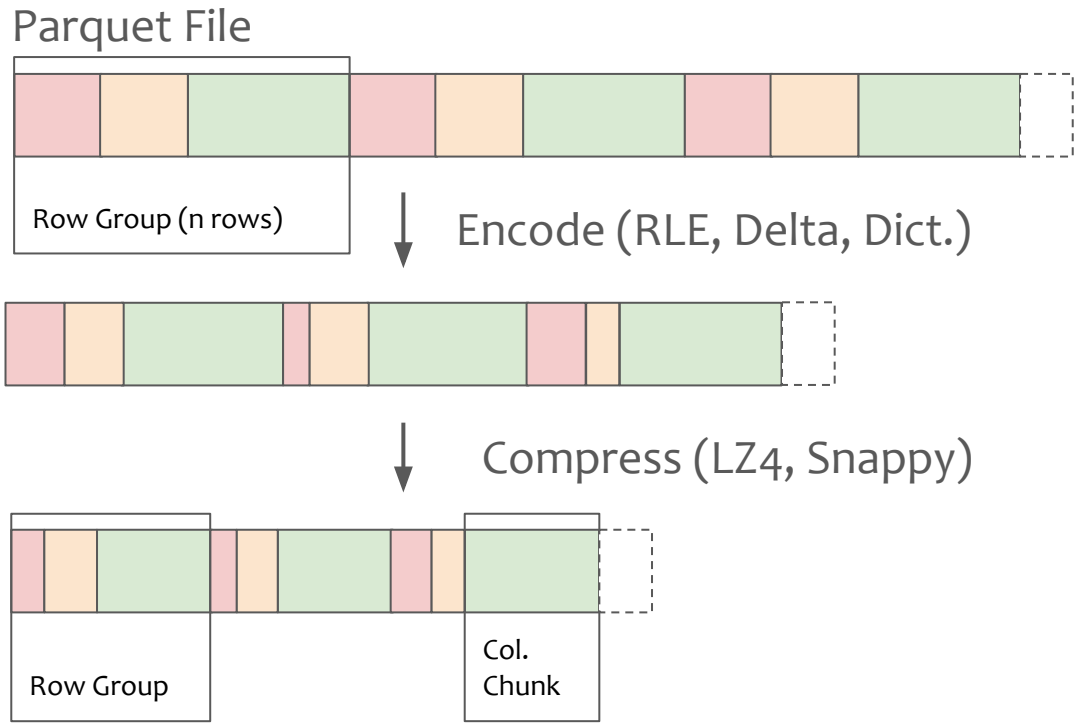


Diagram inspired by "Cloud Analytics Benchmark" paper

Variably-sized regions

Customer ID	Order Number	Comment
XXXX1	1	…
XXXX1	5	…
XXXX2	6	…
…	…	…

Columnar Storage



The resulting file has variably-sized and non-aligned regions.

Research gap

Traditional: Fixed block caching

+ Simple
+ Avoids Fragmentation

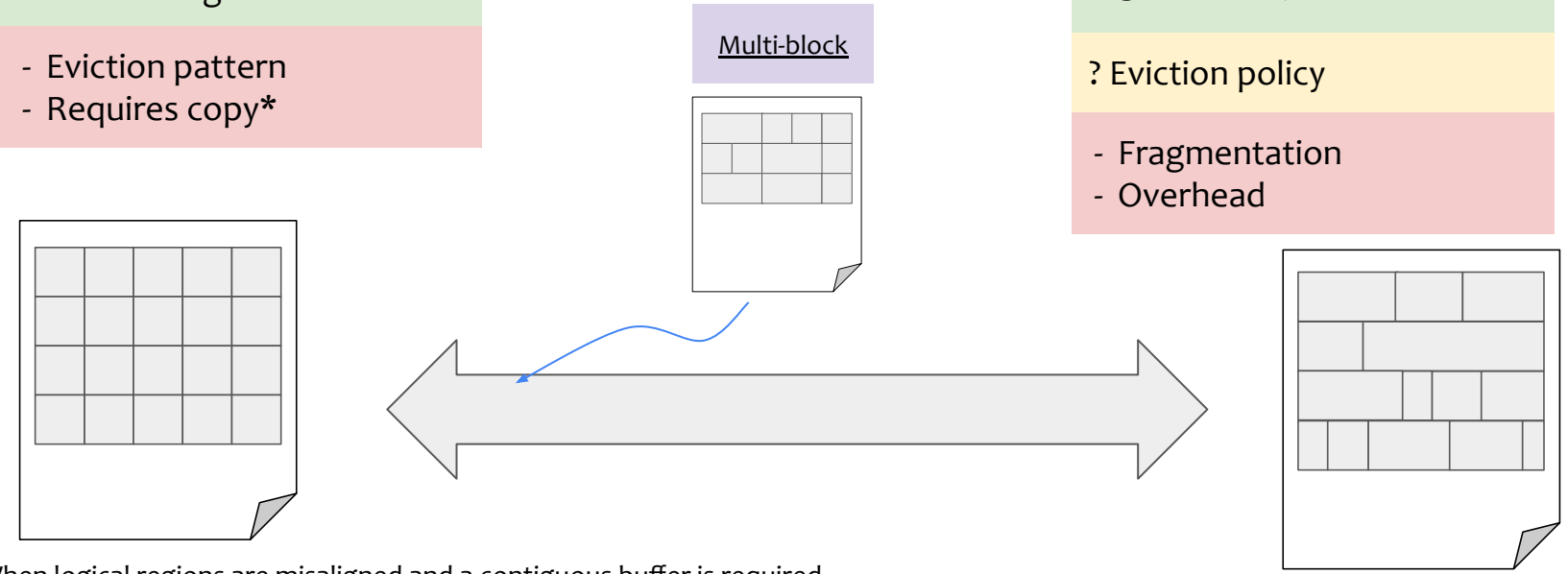
- Eviction pattern
- Requires copy*

Dynamically sized blocks

+ Cache/Eviction on usage granuarality

? Eviction policy

- Fragmentation
- Overhead



*When logical regions are misaligned and a contiguous buffer is required

Problem statement

How can we design a **memory-efficient, variable-sized** buffer cache for cloud analytics applications?

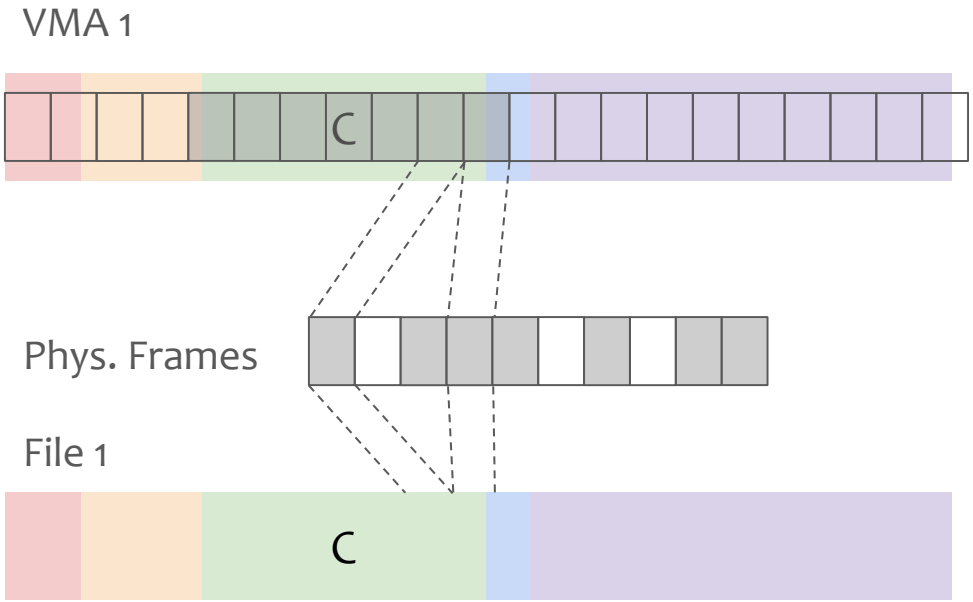
Design overview

- Memory-mapped file
- Leverage VM mapping

+ Cache/Eviction on usage granuarality

Decouple regions/frames

+ Avoid fragmentation



How do we realise this?

We focus only on in-memory caching.

Design overview: Unikernel

- Unikernel, uCache[FAST'26]: memory-mapped I/O
  - uCache assumes fixed-size buffers

- Applications using uCache cannot model variably-sized buffers

Our design leverages a unikernel architecture to enable OS/DB semantic integration.

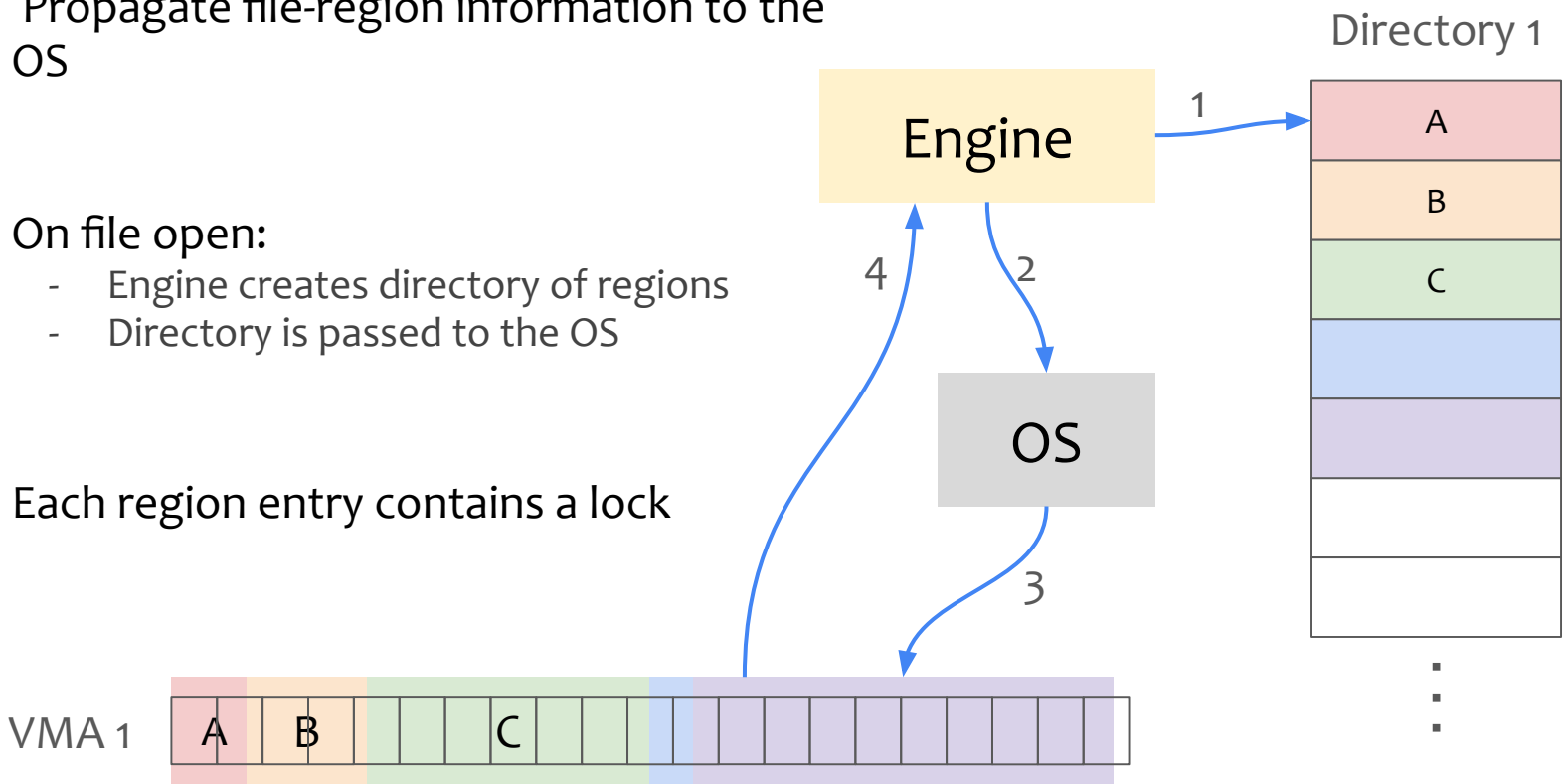
Part of a broader project to enable efficient cloud analytics caching of remote object storage files.

Design: File region directory

Challenge #1: How do we make the buffer manager file-format aware?

Design: File region directory

- Propagate file-region information to the OS
- On file open:
  - Engine creates directory of regions
  - Directory is passed to the OS
- Each region entry contains a lock



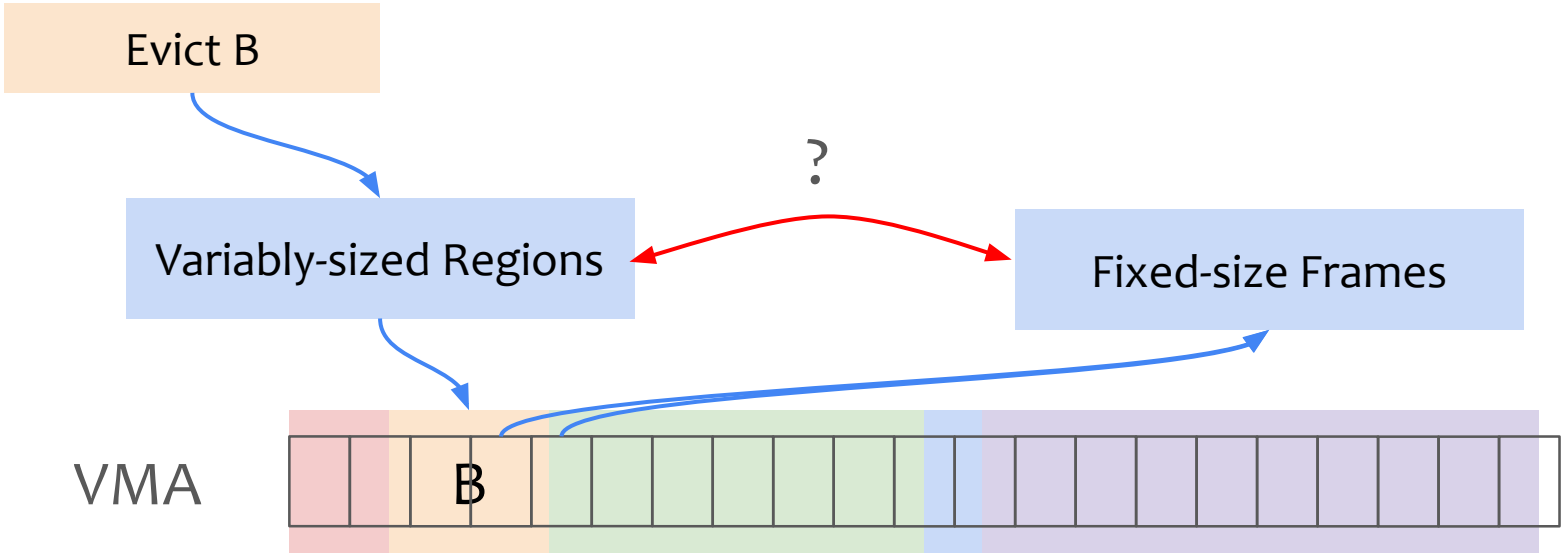
Design: Eviction

Challenge #2: How do we scalably evict variably-sized regions from a memory-mapping?

Challenge #2.1: How do we implement a resident set with variably-sized entries?

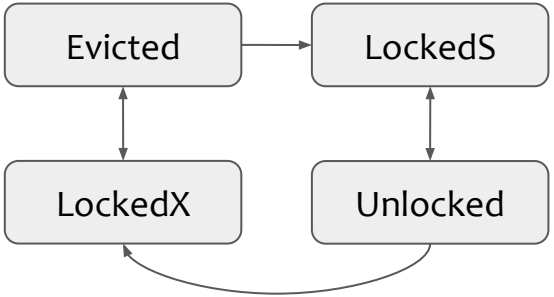
Design: Eviction

- Reconcile logical region eviction with physical frames

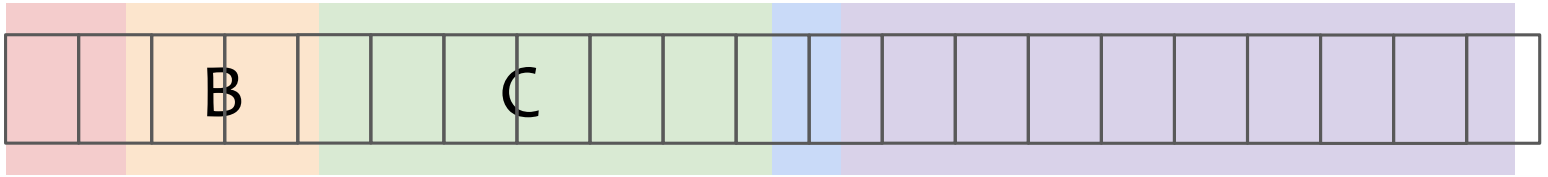


Design: Eviction

- How to handle unaligned regions/frames?
  - Straddling frames
  - Locked adjacent regions
  - Partial region eviction



- Pending I/O?



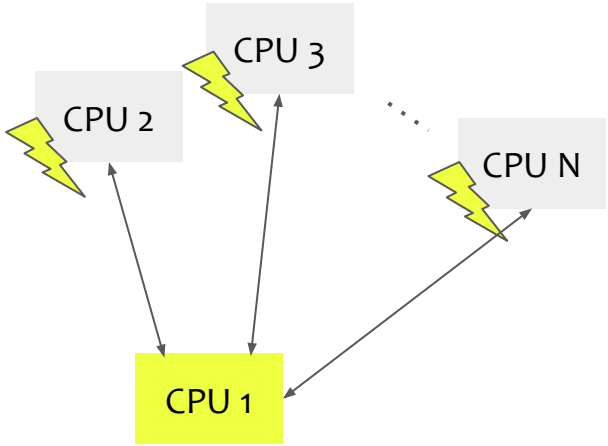
Handle unevictable frames using a *frame* graveyard.

Simplified lock states/transitions.

Design: IPI-free TLB shootdowns

- mmap does not scale to many-core machines

- Eviction: Invalidate TLB entries
  - TLB shootdowns are collective operations
  - Threads must take turns to issue TLB shootdowns



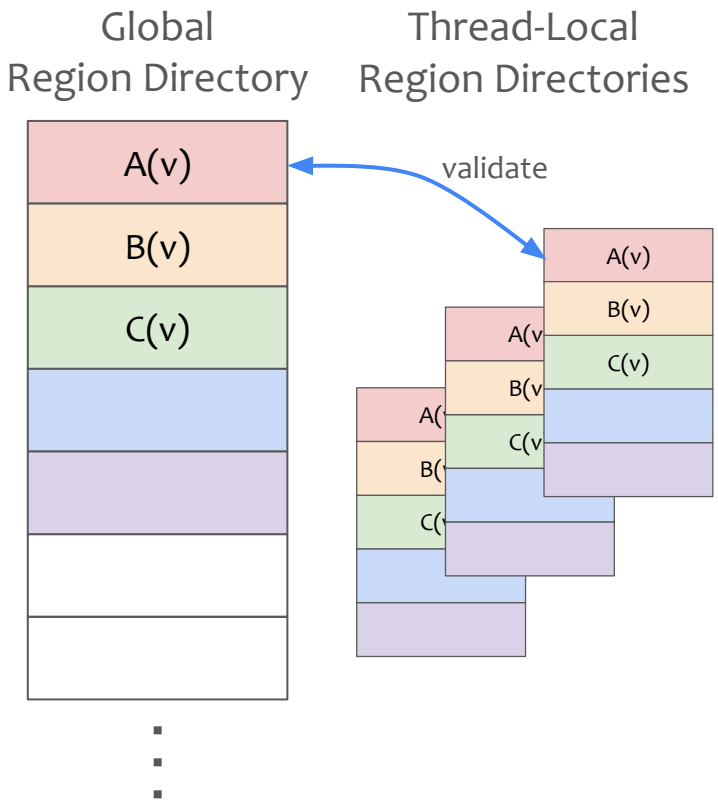
Challenge #3: How do we scale virtual-memory to many-core machines?

IPI: Inter-processor Interrupt. TLB: Translation lookaside buffer.

Design: IPI-free TLB shootdowns

- Embed a version counter in global lock state
- Eviction: increment counter
- Validate TLB entries

TLB invalidations are *lazily* propagated.



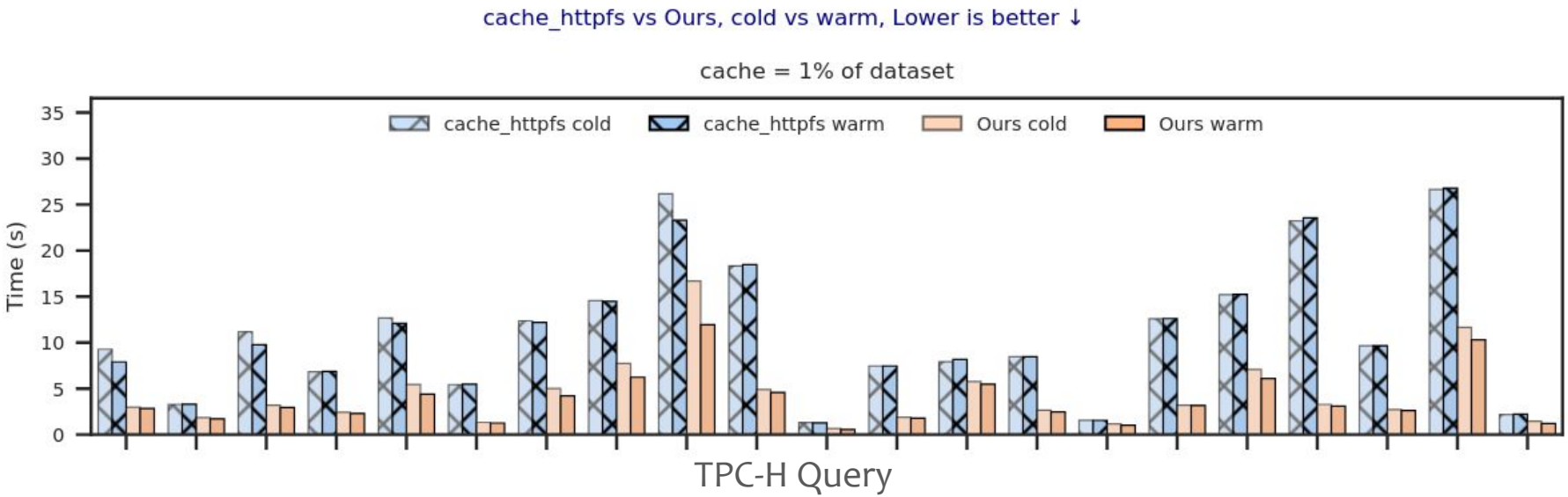
Local directories only need to be core-local, and threads must be core-pinned.

Evaluation: TPC-H cache_httpfs

- Experimental setup:
  - AMD EPYC 9654P CPU (only 16 Cores)
  - Cache: 1% of TPC-H SF100

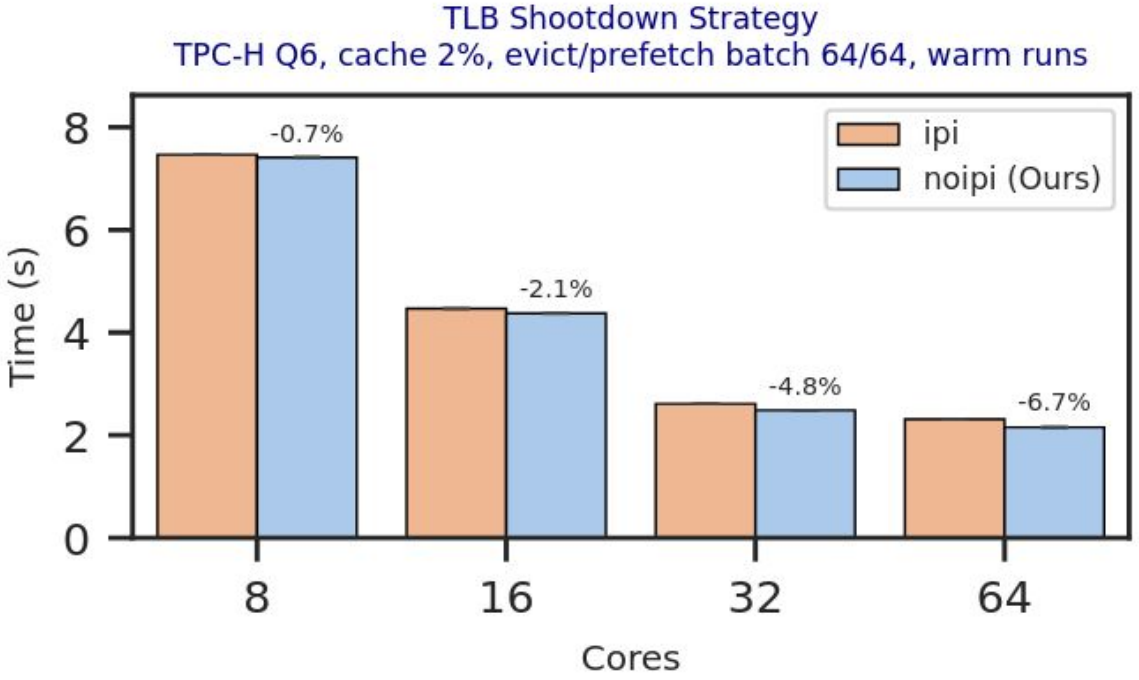
- Baseline:
  - Linux + DuckDB with cache_httpfs extension
  - 512 KiB Block Size, LRU

Evaluation: TPC-H cache_httpfs



Our design achieves a speedup of 2.59× (warm) and 2.85× (cold) over cache_httpfs.

Evaluation: IPI-free TLB shootdown



IPI-free TLB invalidation improves query latency by up to 7%.

Results dependent on exact QEMU thread pinning strategy and batch size

Summary

Current caching solutions primarily operate on fixed-sized chunks

- Fragmentation prevents fine-grained caching
- Memory-mapped I/O approaches are not file-semantics aware
- Virtual-memory based buffer management requires TLB shootdowns

Our Approach:

- Perform cache management on logical file regions
- Reconcile logical regions with physical memory frames
- Avoid TLB shootdowns using lazy invalidation

- Our prototype achieves a speedup of 2.85× over the baseline

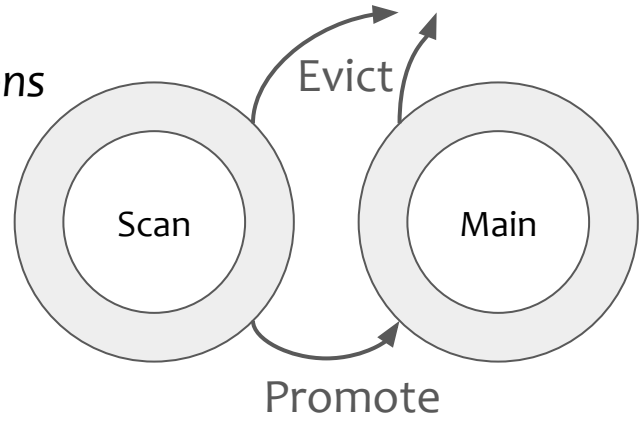
Backup

Design: Eviction policy

- Scale eviction with the resident set
  - Simplify handling of multiple files

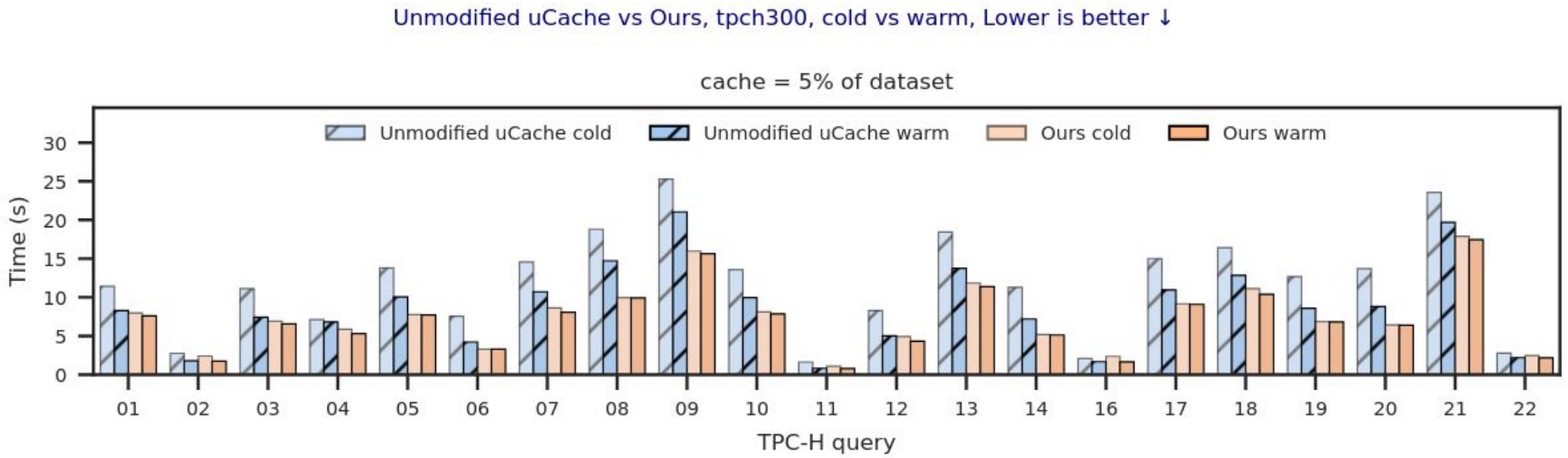
- Resident set entries: ~~Frames~~ -> Variably-sized *Regions*
  - *Entry count* is decoupled from *size* (memory)

- Simplified S3-FIFO (Scan and Main Queue)
  - Second chance



Eviction policy is defined in terms of logical regions.

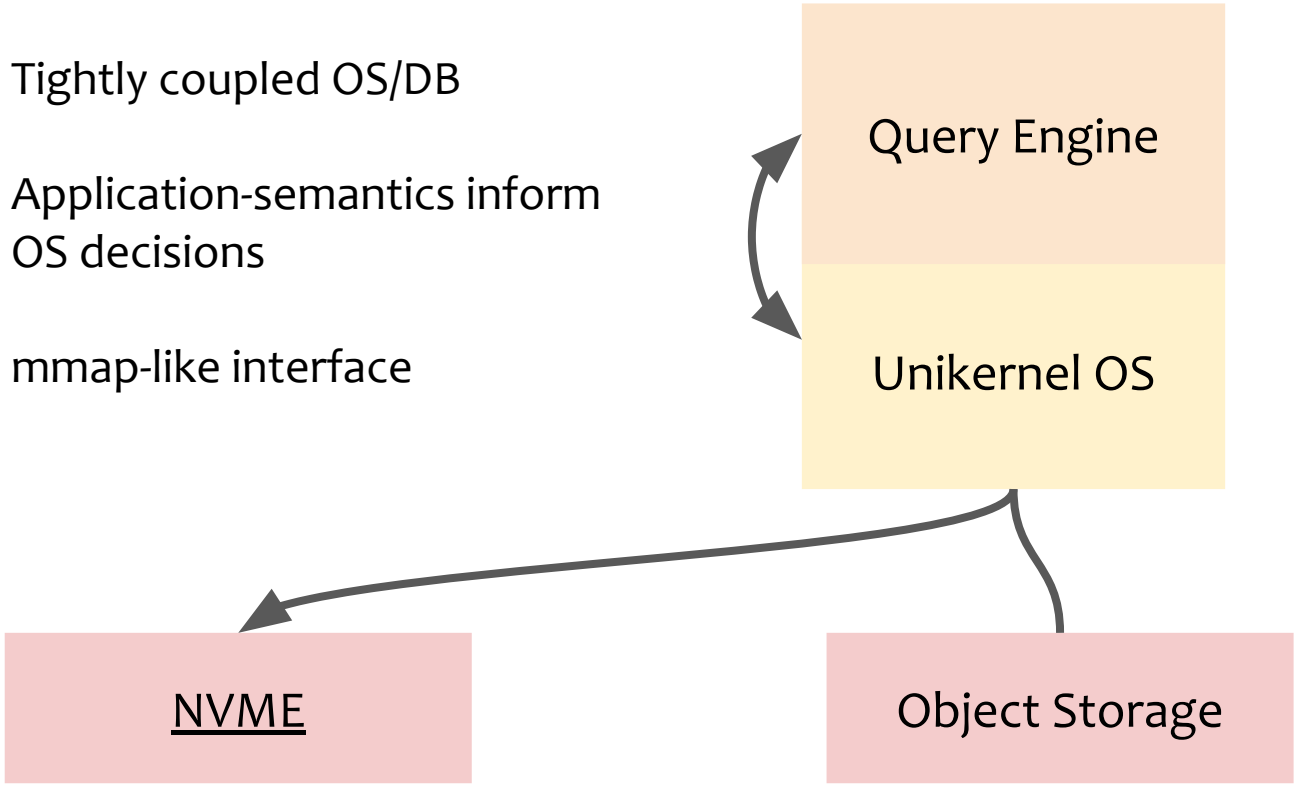
Evaluation: uCache, TPC-H (SF300)



Speedup over uCache: 1.21× (warm), 1.56× (cold)

System overview

- Tightly coupled OS/DB
- Application-semantics inform OS decisions
- mmap-like interface



DuckDB

OSv designed for the cloud

amazon S3

Part of a broader project to enable efficient cloud analytics caching of remote object storage files.